Асинхронный обход ресурсов HTTP-сервера

Предварительные данные

Вам предоставляется исполняемый JAR-файл `Task_J3-server.jar`, который может быть запущен командой

`java -jar Task_J3-server.jar student_id [port]`,

где

- `student_id` — ваши имя и фамилия, записанные подряд латиницей,
- `port` — опциональный целочисленный аргумент, по умолчанию равный `8080`.

После запуска команды стартует HTTP-сервер, использующий указанный порт `port`.

Строка, указанная в `student_id`, используется для генерации входных данных, уникальных для каждого студента.

Примеры сформированных команд для запуска

```
# student_id = IvanIvanov, порт - 8080
java -jar server.jar IvanIvanov
# student-id = PetrPetrov, порт - 3000
java -jar server.jar PetrPetrov 3000
```

При запросе к серверу по методу `GET` с некоторым URL, сервер может ответить сообщением с кодом состояния 200 и

содержимым типа `application/json` следующего формата:

```
{
    "message": "some text",
    "successors": ["path_1", "path_2", "path_3", …, "path_n"],
}
```

где `path_1`, `path_2`, `path_3`, `...`, `path_n` — некоторые строки, такие, что в случае `GET`-запроса к серверу с любой из них в качестве URL-пути, сервер должен вернуть сообщение с кодом состояния 200 и содержимым в представленном выше формате.

Сервер может обрабатывать запрос в течение временного диапазона от 0 до 12 секунд.

Задание

Вам необходимо написать "паука" — программу, которая будет собирать информацию, пересылаемую HTTP-сервером, перемещаясь по путям URL, начиная с корневого — `/`, и продолжая указанным в поле `successors` полученных сообщений.

Результатом работы программы должна быть последовательность сообщений, отсортированная в лексикографическом порядке.

Предполагается использование механизма виртуальных потоков, появившихся в Java 21.

Сервер сконфигурирован таким образом, что информация со всех допустимых URL может быть собрана как минимум за 120 секунд.